Project: Fine-tune a Small Language Model (SLM) for Summarization

Report for CSC_52082_EP at École Polytechnique

Aziz Bacha and Wajdi Maatouk

March 2025

Contents

1 Introduction

This report documents our work for the course **CSC 52082** Introduction to Text Mining and NLP at *École Polytechnique*. The project focuses on fine-tuning a Small Language Model (SLM) for summarization while operating within computational constraints.

Summarization is a key task in *Natural Language Processing (NLP)*, enabling efficient extraction of essential information from large text corpora. While Large Language Models (LLMs) achieve state-of-the-art results, their computational demands make them impractical for many real-world applications. Fine-tuning SLMs offers a balance between efficiency and performance, making them suitable for resource-limited environments.

The objective of this project is to develop a summarization model for the French language. The dataset consists of 5,000 Wikipedia articles, retrieved using the Wikipedia API. The dataset was annotated with synthetic summaries generated by *Qwen2.5-32B-Instruct*, utilizing *LangChain* for structured inference and efficient prompt management. The fine-tuning process was applied to *Qwen2.5-0.5B-Instruct*, using Low-Rank Adaptation (LoRA) and quantization to optimize model performance. The model's performance was evaluated using standard summarization metrics, including ROUGE, BERTScore, and LLM-as-a-Judge, and was compared against the same base model without fine-tuning, serving as the baseline.

This report details the methodology, model training process, evaluation, and findings, as well as optimizations designed to improve the quality and efficiency of the summarization system.

2 Dataset Collection

To build a high-quality dataset for summarization, we collected **5,000 French Wikipedia articles** using the Wikipedia API. The dataset was designed to cover diverse topics, ensuring sufficient linguistic variety for training a summarization model.

2.1 Retrieving Articles from Wikipedia

We used the `wikipedia-api`[3] Python package to retrieve articles from Wikipedia in French. The API was initialized with a specified user-agent to comply with Wikipedia's policies.
To ensure topic diversity, we selected articles from predefined Wikipedia categories, including *Histoire, Science, Culture, Technologie, Économie, Politique, Géographie, Littérature, Sport,* and *Musique.*
Each category was processed recursively to extract articles up to a depth of 10 subcategories, ensuring broad coverage. To optimize performance, articles were collected in parallel using Python's `ThreadPoolExecutor`.

2.2 Filtering and Cleaning Articles

To ensure article quality, the dataset was filtered using the following criteria:

- Articles with fewer than **2,000 characters** were discarded.
- Non-content pages such as disambiguation pages, stubs, and pages containing excessive structured data (e.g., tables, lists) were excluded.
- Only articles from the `MAIN` namespace were considered to exclude meta-information and administrative pages.

After extracting the raw text, we performed additional text cleaning:

- Removal of unwanted sections such as *Notes et Références, Liens externes, Bibliographie*, or sections that did not contain meaningful text.
- Stripping of Wikipedia-specific markup and metadata using regular expressions.

2.3 Parallelized Data Retrieval

To handle the large number of requests efficiently, we implemented **multi-threaded requests** using Python's `ThreadPoolExecutor`, allowing up to 8 concurrent API calls while avoiding rate limits. Additionally, we employed:

- **Exponential backoff**: For API failures, retries were implemented with an increasing delay to prevent request overload.
- **JSON error handling**: Articles that failed due to parsing errors were retried up to 5 times.

The final dataset consists of **5,000 structured French Wikipedia articles**, stored in CSV format for further processing. Each entry contains:

- `Title`: The article title.
- `Summary`: A brief overview of the article.
- `Text`: The cleaned full article content.
- `Category`: The Wikipedia category from which the article was retrieved.

This dataset serves as the foundation for fine-tuning the summarization model.

3 Dataset Annotation

To train a summarization model, we required high-quality reference summaries for our dataset. Since manually annotating 5,000 French Wikipedia articles was impractical, we leveraged a Large Language Model (LLM) to generate synthetic summaries. We selected Qwen2.5-32B-Instruct, an instruction-tuned model known for its strong summarization capabilities.

Given the computational constraints of running a large model, we optimized inference efficiency using 4-bit quantization via the `BitsAndBytes` library. This allowed us to reduce memory usage while maintaining high-quality outputs. Additionally, inference was performed using LangChain[1] to structure the summarization pipeline, ensuring that even longer articles could be processed consistently.

3.1 Summary Generation

Summaries were generated using a structured prompt instructing the model to:

- Provide a clear, concise, and well-structured summary.
- Retain essential information while eliminating unnecessary details.
- Maintain a neutral and factual tone.
- Summarize content across the entire article, including details from later sections.

Inference was computationally expensive, with an average processing time of approximately 20 seconds per article. Given the dataset size, this resulted in a significant overall runtime. However, the generated summaries demonstrated strong coverage of the input texts, successfully capturing key information from all sections, including those appearing toward the end of articles. To mitigate potential system failures, partial results were saved periodically.

3.2 Post-Processing

Once the summaries were generated, we applied a series of post-processing steps to ensure quality and readability:

- Removing unwanted text artifacts such as system-generated prompts and extraneous formatting elements.
- Detecting and eliminating cases where the beginning of the summary was unintentionally repeated.
- Truncating summaries at the last complete sentence to maintain grammatical correctness.

ID	Text	Summary
0	Cet article présente les faits marquants de l'année 1955...	L'année 1955 marque une période importante pour...
1	La caserne Niel est une ancienne caserne militaire...	La caserne Niel, située sur la rive droite de...
2	L'énergie de récupération ou énergie fatale est...	L'énergie de récupération, ou énergie fatale,...
3	Dans le langage économique, un intrant (parfois...	L'article définit un intrant comme un élément...
4	La Characène ou Mésène, était un royaume arabe...	La Characène, également appelée Mésène, était...

Table 1: A preview of the first five rows of the dataset.

Statistic	Text Length	Summary Length
Count	4,659	4,659
Mean	6,137.91	904.39
Standard Deviation	5,499.61	194.32
Minimum	2,000	50
25th Percentile	2,759	853
Median	4,062	929
75th Percentile	7,121.5	1,009
Maximum	48,653	1,325

Table 2: Summary statistics of article and summary lengths.

4 Model Fine-Tuning

4.1 Data Splitting

Before fine-tuning the model, we split the annotated dataset into separate training and test sets. Given that we had 5,000 labeled articles, we allocated:

- **90%** of the data for training.
- **10%** for evaluation, ensuring that the test set contained a representative distribution of article lengths and topics.

During preprocessing, we observed that article lengths varied significantly, ranging from short summaries to very long Wikipedia entries. This posed challenges during training, particularly for longer samples.

4.2 Fine-Tuning the Model

We fine-tuned `Qwen2.5-0.5B-Instruct`, a smaller yet capable instruction-tuned model, using Low-Rank Adaptation (LoRA)[4] to efficiently update model parameters while keeping memory requirements manageable. To further optimize memory usage, we applied `4-bit quantization`[2] using the `BitsAndBytes` library.

The fine-tuning setup included:

- **LoRA configuration:** Rank of 32, alpha of 64, and dropout of 0.05.
- **Tokenization:** The model used a tokenizer with EOS padding to ensure consistency across training batches.
- **Training strategy:** Articles were tokenized with only the summary contributing to the loss function, ensuring the model learned the summarization task correctly.
- **Hyperparameters:** A learning rate of 3×10^{-4}, batch size of 1 (to prevent memory issues), gradient accumulation of 16, and training for 3 epochs.

We trained the model using `Hugging Face's Trainer` framework, which allowed for efficient checkpointing and logging via TensorBoard.

4.3 Memory Constraints and Adaptive Fine-Tuning

A major challenge during fine-tuning was encountering **CUDA out-of-memory (OOM)** errors when processing long articles. Even with quantization and LoRA applied, the longest articles exceeded GPU memory capacity, leading to frequent crashes. To mitigate this, we implemented a chunk-based fine-tuning strategy:

- The dataset was **sorted by article length** and split into **10 chunks**, from the shortest articles to the longest.
- Fine-tuning was performed **sequentially**, processing each chunk individually while ensuring training stability.
- Despite this approach, **chunks 9 and 10** consistently triggered OOM errors and could not be used.

To compensate for the unusable longer articles and introduce more variety into the dataset, we retrieved 1,000 additional shorter articles from the MLSUM (MultiLingual SUMmarization) dataset (Scialom et al.)[6], reran the summarization pipeline, and reintegrated them into the dataset. This adjustment enabled us to meet the required number of training samples while mitigating memory constraints.

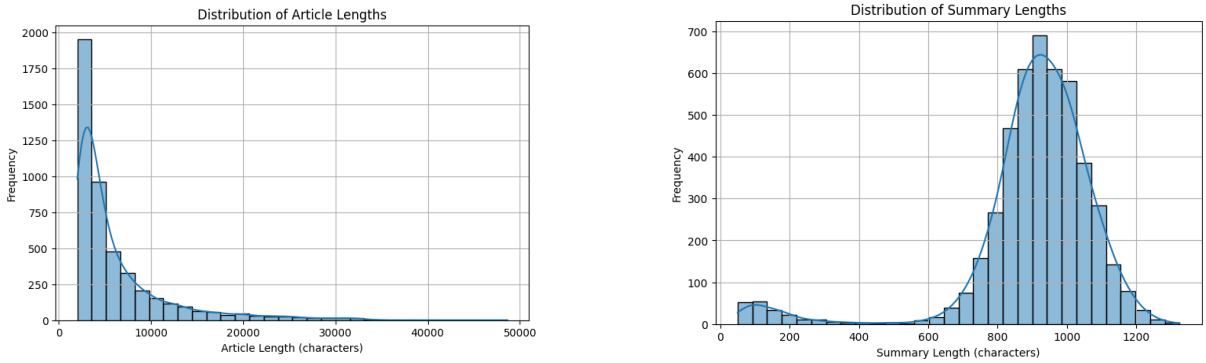

Figure 1: Distribution of article and summary lengths.

5 Model Evaluation

Assessing the performance of summarization models is particularly challenging, as it involves multiple factors such as factual accuracy, fluency, coherence, and the extent of information coverage. This section outlines our evaluation strategy and presents a comparison between our fine-tuned model and the baseline.

5.1 Evaluation Metrics

5.1.1 ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)[5] is a widely used metric for summarization evaluation. It measures the overlap of n-grams and sequences between the generated summaries and the reference summaries. The main ROUGE scores reported include:

- **ROUGE-1**: Measures unigram (word-level) overlap.
- **ROUGE-2**: Measures bigram (two-word sequence) overlap.
- **ROUGE-L**: Measures the longest common subsequence (LCS) between the predicted and reference summaries.

Higher ROUGE scores indicate better lexical similarity between the generated and reference summaries.

5.1.2 BERTScore

BERTScore[7] is a more semantically aware metric that leverages contextual word embeddings from transformer models. Instead of exact n-gram overlap, it computes the cosine similarity between word representations in a high-dimensional space, allowing for a better evaluation of paraphrasing and synonymy.

BERTScore is particularly useful in abstractive summarization tasks where the model may generate valid summaries with different word choices than the reference.

5.2 Analysis of Model Evaluation

The evaluation results in Figure 2 show that fine-tuning significantly improved summarization performance across all metrics—ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore. The fine-tuned model consistently outperforms the base model, achieving higher median scores and a more stable distribution.

The ROUGE metrics indicate that the fine-tuned model retains more key information from the original text while reducing hallucinations and inconsistencies. The increase in BERTScore further suggests that its summaries are more semantically aligned with the references. Notably, the fine-tuned model performs close to the *Qwen-32B model*, despite having a much smaller number of parameters.

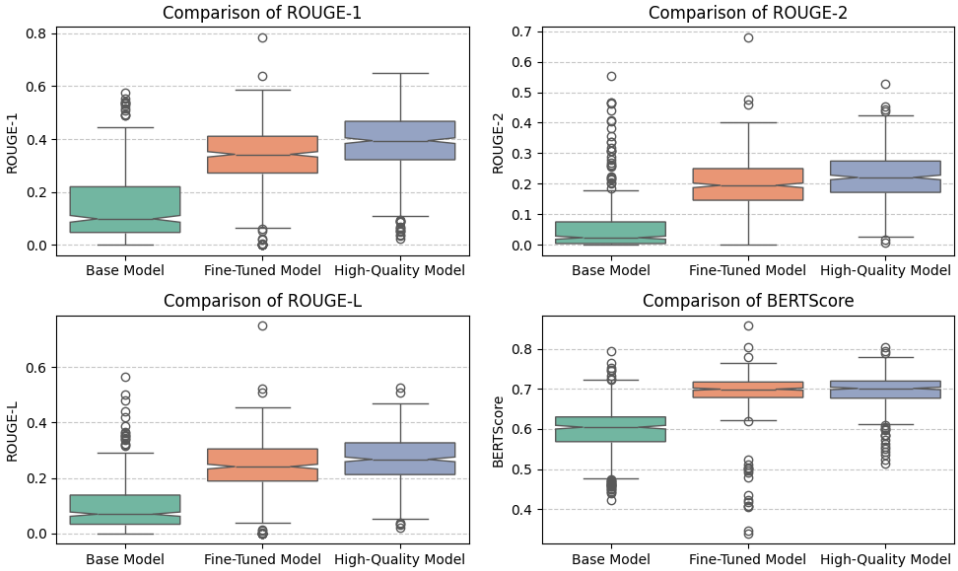

Figure 2: Comparison of ROUGE and BERTScore metrics for the base, fine-tuned, and high-quality models.

We also examined the summaries ourselves and noticed a clear improvement with the fine-tuned model. Compared to the baseline, its summaries were more accurate, avoided hallucinations, and contained fewer repeated sentences. The model better captured key information while maintaining a natural and coherent flow. To illustrate this, we included an example of such summaries in the annex.

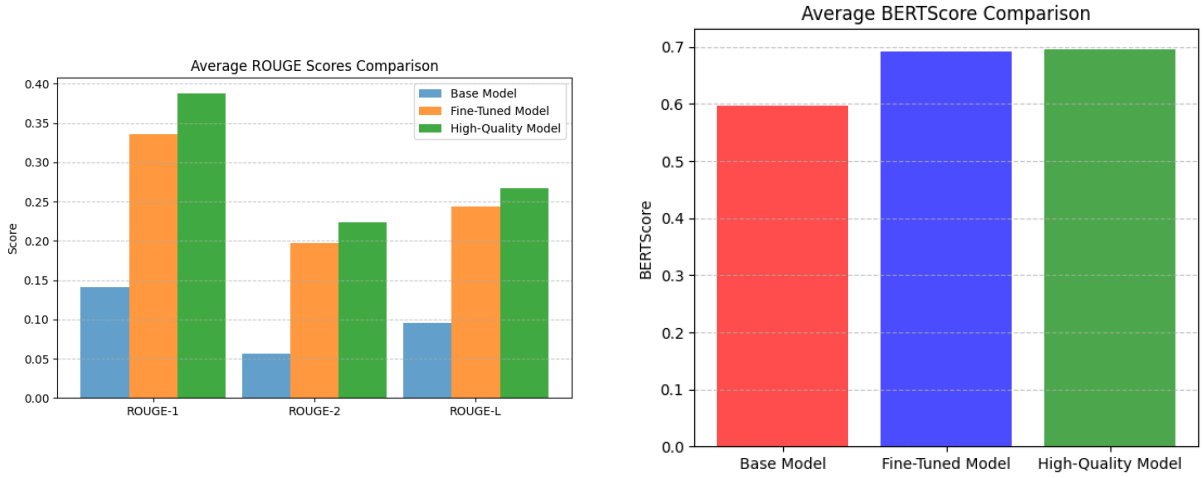

Figure 3: Comparison of average ROUGE and BERTScore metrics across models. The fine-tuned model shows significant improvement over the base model, reaching performance close to the high-quality reference model.

References

[1] Harrison Chase. *LangChain: A Framework for Developing Applications with LLMs*. 2023. URL: https://github.com/hwchase17/langchain.

[2] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *NeurIPS* (2023). URL: https://arxiv.org/abs/2305.14314.

[3] Wikipedia API Developers. *Wikipedia API for Python*. 2023. URL: https://github.com/martin-majlis/Wikipedia-API.

[4] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)* (2022). URL: https://arxiv.org/abs/2106.09685.

[5] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out* (2004), pp. 74–81. URL: https://aclanthology.org/W04-1013/.

[6] Thomas Scialom et al. "MLSUM: The Multilingual Summarization Corpus". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, Nov. 2020, pp. 8051–8067. DOI: 10.18653/v1/2020.emnlp-main.647. URL: https://aclanthology.org/2020.emnlp-main.647.

[7] Tianyi Zhang et al. "BERTScore: Evaluating Text Generation with BERT". In: *International Conference on Learning Representations (ICLR)* (2020). URL: https://arxiv.org/abs/1904.09675.

6 Annex

6.1 Example of a Summary Generated by the Baseline Model vs. the Fine-Tuned Model

Summary of the baseline model:

> Dans la mythologie grecque, Léda est une figure importante, car elle est l'épouse de Tyndare et la mère de Clytemnestre, Hélène, Pollux et Castor. Son rôle est important dans la mythologie grecque, car il est considéré comme étant le père de Zeus, le dieu du solstice.
>
> Ce mythe est également représenté dans diverses formes de musique, notamment dans les œuvres de musique classiques, comme *Swan upon Leda*, de Hozier, qui est une œuvre de musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne,

qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne, qui est une musique classique moderne.

Summmary of the fine-tuned model:

Dans la mythologie grecque, Léda, fille du roi Thestios, est l'épouse du roi Tyndare et la mère de divers enfants. Son mythe relate l'histoire où Zeus, sous forme d'un cygne, séduit Léda par ruse avant de concevoir deux enfants : Hélène et Pollux, et Clytemnestre et Castor. D'autres versions diffèrent sur le rôle de Dioscures ou de Tyndare. Le mythe a inspiré de nombreux artistes et peintres, notamment en littérature, musique et sculpture. Parmi les œuvres notables, on trouve des époques importantes comme celles de Pierre de Ronsard, Alfred Jarry, et Gustave Moreau. Les représentations modernes incluent des sculptures, des peintures et des films, reflétant ainsi son importance culturelle et mythologique.